

Report 03

Infrastructure Scaffolding & Data Pipeline Architecture

Project:	JalJeev — Agentic Marine Intelligence System
Phase:	Phase 0, Week 1 (Aug 25–31, 2026)
Focus:	Repository scaffold, Docker infrastructure, database schema, initial adapters

1. Objective

Stand up the foundational infrastructure: version-controlled repository, containerised services (PostgreSQL+PostGIS, Redis, MinIO), database schema, and the first two data-source adapter templates (INCOIS ERDDAP, IMD) — everything the agent layer needs to exist before the first agent can run.

2. Repository Structure Established

```
JalJeev/  
├── backend/  
│   ├── app/  
│   │   ├── agents/      # Agent implementations  
│   │   ├── api/         # FastAPI routers  
│   │   ├── core/        # Config, cache, Celery  
│   │   ├── db/          # SQLAlchemy async  
│   │   ├── ml/          # ML pipeline  
│   │   ├── models/      # Pydantic schemas  
│   │   ├── planner/     # LangGraph state machine  
│   │   └── tools/       # Data adapters  
│   ├── frontend/       # React + Vite + TypeScript  
│   ├── android/        # Kotlin, Jetpack Compose  
│   ├── infra/sql/      # PostgreSQL schema  
│   ├── scripts/        # Data acquisition  
│   ├── data/           # Local cache (gitignored)  
│   └── docs/           # Documentation  
├── docker-compose.yml  
├── .env.example  
└── ORCA_PRD_SIH26176_v2.md
```

3. Docker Compose Infrastructure

3.1 Services Provisioned

Service	Image	Port	Purpose
postgres	postgis/postgis:16-3.4	5433 (host) → 5432	Primary DB with PostGIS
redis	redis:7.2-alpine	6379	Cache + Celery broker + session
minio	minio/minio:latest	9000, 9001	S3-compatible object storage

backend	Custom (Uvicorn)	8000	FastAPI application server
celery_worker	Same image	—	Background task execution
celery_beat	Same image	—	Scheduled ingestion jobs

3.2 Key Design Decision: Postgres on Port 5433

The development machine already had another PostgreSQL instance on the standard port 5432. Rather than conflict, we mapped JalJeev's Postgres to port **5433**. The .env file's DATABASE_URL points at localhost:5433 for local dev; docker-compose.yml overrides it to the internal postgres:5432 hostname for containerised services.

4. Database Schema (Phase 0)

4.1 Initial Tables

Table	Purpose	Key Columns
users	User accounts + vessel profile	id, language_pref, home_port
vessel_profiles	Vessel physical characteristics	length_m, draft_m, max_safe_wave_m, operational_range_km
h3_marine_states	H3-indexed per-cell marine state vectors	h3_cell, sst_c, wave_height_m, wind_speed_ms, risk_score
ingestion_log	Data pipeline audit trail	source, dataset_id, status, records_ingested, error_msg
decisions	Audit trail for every recommendation	user_id, query, response, risk_level, sources_used, created_at
ports	Indian port coordinates	name, lat, lon, aliases

4.2 PostGIS Extensions

Enabled postgis and postgis_topology for spatial queries — critical for EEZ containment checks, MPA intersection tests, and nearest-port queries using ST_DWithin, ST_Contains, ST_Distance.

5. Configuration Management

5.1 Settings Architecture

Used Pydantic BaseSettings to load all configuration from environment variables:

```
class Settings(BaseSettings):
    DATABASE_URL: str
    REDIS_URL: str

    # LLM (optional)
    GEMINI_API_KEY: str | None = None
    GROQ_API_KEY: str | None = None

    # Tier 1 data sources
    MOSDAC_USERNAME: str | None = None
    IMD_API_KEY: str | None = None
    COPERNICUS_MARINE_USERNAME: str | None = None

    # Public endpoints (no auth)
    INCOIS_ERDDAP_URL: str = "https://erddap.incois.gov.in/erddap"
    NOAA_ERDDAP_URL: str = "https://coastwatch.pfeg.noaa.gov/erddap"
    OPEN_METEO_MARINE_URL: str = "https://marine-api.open-meteo.com/v1/marine"
```

Design principle: Every credential defaults to None. An adapter whose credential is missing returns {"status": "unavailable"} — dropping the key into .env activates it with zero code changes. This means the system works from day one, degraded but honest, and data sources come online incrementally.

5.2 Environment Template

Created a fully annotated 71-line .env.example documenting every variable, where to register for it, and expected activation time (immediate, 1–3 days, 2–5 days).

6. First Data Adapters

6.1 INCOIS ERDDAP Adapter

Protocol: ERDDAP (public, no authentication)

Library: erddapy

Data: Wave height, wave period from Ocean State Forecast

Pattern: async function wrapping a blocking erddapy call in a dedicated thread executor

Key note: dataset_id = "incois_osf_wave" was a placeholder — flagged with TODO(data-eng) for verification against the live catalog

6.2 IMD Adapter

Protocol: REST API (requires IMD_API_KEY)

Library: httpx (async HTTP client)

Data: Weather observations and forecasts

Pattern: Returns {"status": "unavailable", "reason": "IMD_API_KEY not configured"} when the key is missing

Key note: IMD does not publish a single stable public REST API — the adapter isolates this uncertainty behind one function

6.3 Adapter Contract

Every adapter returns a dict with at minimum:

```
{"status": "success" | "failed" | "unavailable", "source": "<name>"}
```

- "success" — data retrieved and returned
- "failed" — tried and encountered an error
- "unavailable" — not configured yet

7. Thread Pool Architecture

A dedicated `ThreadPoolExecutor(max_workers=6, thread_name_prefix="net")` was created specifically for blocking network calls. This was a proactive design decision based on knowing that `asyncio.to_thread()` shares `asyncio`'s internal pool and would become a bottleneck — which was later confirmed live.

8. FastAPI Skeleton

The initial `app/main.py` provided:

- `/health` — liveness check
- `/debug/incois/wave?lat=X&lon;=Y` — sanity-check endpoint for the INCOIS adapter
- `/debug/imd/weather?lat=X&lon;=Y` — sanity-check endpoint for the IMD adapter

These debug endpoints let the team confirm data connectivity before the agent layer existed.

9. Week 1 Deliverables Summary

Deliverable	Status
Git repository initialised	■
docker-compose.yml with all services	■
PostgreSQL + PostGIS schema deployed	■
Redis running	■
.env.example with all credentials	■
INCOIS ERDDAP adapter template	■
IMD adapter template	■
FastAPI skeleton with debug endpoints	■
Pydantic settings configuration	■
Dedicated network executor pool	■

10. Risks Identified

Risk	Mitigation
INCOIS ERDDAP dataset IDs are unverifiable on the web	Flagged on the UI; verify against live catalog
IMD has no stable public API documentation	Adapter isolates the endpoint; fallback to Open-Meteo
MOSDAC registration pending	Adapters designed to activate on credential drop-in

Report prepared by Team ORCA, KIIT University, Bhubaneswar